

Table of Contents

1.	Introduction & Company Overview	3
1.1.	Introduction	3
1.2.	Company Overview	3
1.3.	Department Overview	4
1.4.	Motivation for the Project	4
2.	Role & Tasks	4
3.	Project Overview	6
4.	Tools & Frameworks	7
4.1.	Laravel	8
4.2.	PostgreSQL and Vector Database Extension	8
4.3.	Embedding Model	8
4.4.	Language Model	9
4.5.	Tailwind CSS	9
4.6.	Blade Template Engine	9
4.7.	JavaScript	9
4.8.	Document Text Extraction	10
4.9.	Optical Character Recognition (OCR)	10
4.10.	Background Job Processing	10
4.11.	Docker	10
4.12.	Git and GitHub	10
4.13.	Development Environment	11
5.	Specification	11
5.1.	Purpose, Scope, Users, and Constraints	11
5.2.	Functional Requirements	12
5.3.	Non-functional Requirements	14
5.4.	User Stories	15
5.5.	Use Case Diagrams and Use Case Descriptions	18
5.6.	External Interfaces	25

5.6.1.	Database Interface	25
5.6.2.	Embedding Model Interface.....	25
5.6.3.	Language Model Interface	26
5.6.4.	Background Processing Interface.....	26
5.6.5.	Containerization Interface	27
5.7.	User Interface Mockups.....	27
5.7.1.	Login Interface.....	28
5.7.2.	User Dashboard	28
5.7.3.	Document Upload Interface.....	29
5.7.4.	Document Details and Processing Status	30
5.7.5.	Conversation Interface	30
5.7.6.	Conversation History	31
5.7.7.	Administrative Dashboard	32
6.	Design	32
6.1.	System Architecture	32
6.2.	Component Diagram	34
6.3.	Class Diagram	36
6.4.	Sequence Diagrams.....	37
6.4.1.	Document Upload & Processing	38
6.4.2.	Question Answering Workflow	39

1. Introduction & Company Overview

1.1. Introduction

As part of the Master's program in Software Engineering for Industrial Applications at Hof University of Applied Sciences, students are required to apply theoretical knowledge in a practical industrial environment through project-based work. This report documents the software specification and design activities performed during the development of an AI-powered Retrieval-Augmented Generation (RAG) system for enterprise document management and knowledge retrieval.

Modern organizations generate and maintain large volumes of technical documents, reports, manuals, process descriptions, and operational guidelines. Accessing relevant information from these documents often requires significant manual effort, leading to reduced efficiency and increased time spent searching for knowledge. To address this challenge, an intelligent document retrieval system was designed and developed that combines document processing, semantic search, and Artificial Intelligence technologies to provide context-aware answers based on organizational documents.

The project was developed during my work as a Working Student at Siemens Energy. The development process involved requirements analysis, software specification, system design, architecture planning, implementation, and validation of the proposed solution. This report focuses on the specification and design aspects of the system, including functional and non-functional requirements, use cases, system architecture, interface design, and UML models.

The objective of the project was to develop a scalable and maintainable software solution that can process organizational documents, retrieve relevant information, and help users find answers through natural language queries.

1.2. Company Overview

Siemens Energy is one of the world's leading energy technology companies, operating across the entire energy value chain. The company develops innovative solutions for power generation, transmission, storage, and industrial applications while supporting the global transition toward sustainable and reliable energy systems.

With operations in more than 90 countries and a workforce of approximately 100,000 employees worldwide, Siemens Energy serves customers in both conventional and renewable energy sectors. The company provides technologies and services for gas turbines, steam turbines, power generation systems, grid infrastructure, hydrogen solutions, industrial applications, and digital transformation initiatives.

Digitalization plays a critical role in Siemens Energy's strategy. The company continuously invests in advanced software solutions, data-driven decision making, artificial intelligence, cloud technologies, and process automation to improve operational efficiency and support business innovation. These initiatives enable employees and stakeholders to access information more effectively, optimize workflows, and enhance organizational productivity.

1.3. Department Overview

The project was conducted within the SE GT DG OE department of Siemens Energy.

The department is involved in supporting digitalization initiatives, process optimization, and data-driven solutions that enhance operational efficiency across business functions. Various software tools, reporting systems, and analytical platforms are utilized to manage information, automate processes, and improve collaboration among stakeholders.

As part of ongoing digital transformation efforts, the department actively explores modern technologies such as cloud platforms, data analytics, artificial intelligence, and enterprise software systems. These technologies are used to streamline business processes, improve knowledge management, and support informed decision-making.

The development of the AI-powered document retrieval system aligns with these objectives by providing an intelligent mechanism for managing and accessing organizational knowledge. The project contributes to the broader goal of improving information accessibility and reducing the effort required to locate relevant content within large collections of enterprise documents.

1.4. Motivation for the Project

Organizations frequently store critical knowledge across numerous documents, including technical manuals, operational procedures, project documentation, and internal reports. Although these documents contain valuable information, retrieving specific content often requires manual searching, which can be time-consuming and inefficient.

The increasing adoption of Artificial Intelligence technologies has created opportunities to enhance document management systems through semantic understanding and context-aware information retrieval. Instead of relying solely on keyword-based searches, modern AI-assisted systems can understand user intent and identify relevant information based on contextual meaning.

The motivation behind this project was to design and develop a software solution capable of improving access to organizational knowledge while maintaining a user-friendly experience. By integrating document processing, semantic retrieval mechanisms, and AI-assisted response generation, the proposed system aims to support employees in obtaining accurate information quickly and efficiently.

The project also provided an opportunity to apply software engineering principles, including requirements engineering, system design, architectural modeling, and software development methodologies within a real industrial environment.

2. Role & Tasks

During my internship at Siemens Energy, I worked as a Working Student in the Digital Solutions team, where I contributed to both enterprise software development and data engineering activities. My responsibilities were divided between supporting internal data analytics initiatives and developing an AI-powered Retrieval-Augmented Generation (RAG) system for enterprise document management and

knowledge retrieval. Throughout the internship, I collaborated with experienced engineers and stakeholders to understand business requirements, design software components, implement new features, and validate the developed solutions.

My primary responsibility was the design and development of the RAG system. The objective of this project was to provide employees with a secure and efficient way to retrieve information from enterprise documents using semantic search and AI-assisted question answering. I participated in the complete software development lifecycle, including requirement analysis, system design, backend implementation, user interface development, testing, debugging, and documentation. The system was designed with scalability, maintainability, and usability as key objectives.

To support document processing, I implemented functionality for uploading and managing enterprise documents through a web-based interface. The processing pipeline was designed to automatically extract document text, perform preprocessing, divide the content into semantic chunks, generate vector embeddings, and store them in a vector-enabled database for efficient retrieval. Additional document processing capabilities, including optical character recognition for scanned documents and support for multiple document formats, were incorporated to improve the robustness of the system.

Another important responsibility involved developing the retrieval and conversational components of the application. I implemented semantic search to retrieve the most relevant document fragments based on user queries and integrated these results with an AI model to generate context-aware responses. The chat interface was designed to support conversation history, multiple document selection, citation of retrieved information, and an intuitive workflow for interacting with enterprise knowledge.

In addition to implementing application features, I continuously improved the overall software architecture by optimizing database queries, refining background processing workflows, and enhancing the responsiveness of the user interface. I also contributed to deployment preparation by containerizing the application and documenting the development environment to simplify collaboration and future maintenance.

Alongside the RAG project, I contributed to enterprise data engineering tasks within Siemens Energy. My work included developing and validating data processing workflows using Python, SQL, and Snowflake, transforming business data for reporting purposes, and supporting the development of Power BI dashboards. These activities involved data quality verification, optimization of data transformation logic, and collaboration with domain experts to ensure that analytical reports accurately reflected business requirements.

Throughout the internship, I regularly participated in team discussions, code reviews, and technical knowledge sharing sessions. Feedback received during these activities was incorporated into subsequent development iterations, resulting in continuous improvements to both the software quality and development process. I also prepared technical documentation describing the implemented components, development workflow, and deployment procedures to support future system maintenance and further enhancements.

The combination of enterprise software development, AI-assisted document retrieval, and data engineering allowed me to strengthen both my technical and professional skills. Working on real-world industrial projects provided valuable experience in designing scalable software systems, collaborating within multidisciplinary teams, and delivering solutions that address practical business challenges while following professional software engineering practices.

3. Project Overview

The project developed during this internship is an AI-powered Retrieval-Augmented Generation (RAG) system designed to improve enterprise document management and knowledge retrieval. The application enables users to upload organizational documents, process their contents automatically, and interact with them through a conversational interface. Instead of manually searching through lengthy reports or technical documentation, users can ask questions in natural language and receive responses generated from the relevant information contained within the uploaded documents.

The primary objective of the proposed system is to reduce the time required to locate information while ensuring that responses are generated from trusted organizational documents rather than relying solely on the knowledge of a language model. By combining semantic document retrieval with AI-assisted response generation, the system provides more accurate, context-aware, and document-grounded answers. This approach also helps reduce unnecessary processing by supplying only the most relevant document content to the AI model instead of entire documents.

Document Upload and Management

The system provides an interface for uploading and managing enterprise documents. Each uploaded document is associated with metadata that allows users to organize and monitor the processing status throughout its lifecycle. The application supports multiple document formats commonly used within organizations, allowing users to maintain a centralized repository of documents that can later be queried through the conversational interface.

Automated Document Processing

After a document is uploaded, the system automatically executes a processing pipeline that prepares the document for semantic retrieval. The document content is extracted, cleaned, and divided into smaller semantic sections that preserve contextual meaning while remaining suitable for efficient retrieval. This automated workflow minimizes manual intervention and ensures that newly uploaded documents become searchable after processing is completed.

Semantic Knowledge Retrieval

Rather than relying on traditional keyword-based searching, the system performs semantic retrieval to identify document sections that are most relevant to the user's query. This allows the application to retrieve information based on contextual similarity instead of exact word matching. As a result, users can ask questions using natural language without needing to know the precise terminology used within the original documents.

AI-Assisted Question Answering

Once the relevant document sections have been identified, they are provided as contextual information to an AI model responsible for generating the final response. Since the generated answer is based on retrieved document content, the system produces responses that remain grounded in the available enterprise knowledge. The application also presents references to the source documents, allowing users to verify the information and explore the original context when necessary.

Conversation-Based User Interaction

The system provides a conversation-oriented interface that enables users to interact naturally with their uploaded documents. Users can create multiple conversations, select one or more documents for each discussion, review previous conversations, and continue asking follow-up questions while maintaining the context of earlier interactions. This conversational workflow provides a more intuitive alternative to repeatedly performing independent document searches.

Scalable and Extensible Architecture

The developed solution has been designed with scalability and maintainability in mind. Its modular architecture separates document management, document processing, semantic retrieval, conversational interaction, and background processing into independent components. This separation simplifies future enhancements, allowing additional document formats, retrieval strategies, administrative capabilities, and AI-assisted features to be incorporated with minimal impact on the existing system.

Overall, the developed RAG system provides an intelligent approach to enterprise knowledge retrieval by combining automated document processing, semantic search, and AI-assisted question answering within a single integrated platform. The system improves information accessibility, reduces the effort required to locate relevant knowledge, and offers employees an efficient way to interact with enterprise documentation while maintaining the reliability of document-based responses.

4. Tools & Frameworks

The development of the AI-powered Retrieval-Augmented Generation (RAG) system required the integration of several frameworks, development tools, and supporting technologies. Each technology was selected based on its suitability for building a scalable, maintainable, and efficient enterprise web application capable of document processing, semantic retrieval, and AI-assisted question answering.

Tool / Framework	Purpose
Laravel	Backend web application framework
PostgreSQL	Relational database management system
Vector Database Extension	Storage and similarity search of document embeddings
Embedding Model	Generation of semantic vector embeddings
Language Model	Generation of context-aware responses
Tailwind CSS	Frontend styling and responsive user interface

Blade Template Engine	Server-side view rendering
JavaScript	Interactive frontend functionality
Document Text Extraction Tool	Extraction of text from PDF documents
Optical Character Recognition (OCR)	Extraction of text from scanned documents
Docker	Application containerization
Git	Version control
GitHub	Source code management and collaboration
Visual Studio Code	Integrated development environment
Composer	PHP dependency management
Node.js & npm	Frontend package management and asset compilation

Table 4.1: provides an overview of the main technologies used throughout the development of the project.

4.1. Laravel

Laravel was used as the primary backend framework for developing the application. It follows the Model-View-Controller (MVC) architectural pattern, which helps organize the application into well-structured components and promotes maintainable software development.

The framework provides built-in features such as routing, authentication, middleware, database abstraction, background job processing, and dependency injection. These capabilities simplified the implementation of document management, user authentication, background processing, and communication between different parts of the system while maintaining a clean and modular architecture.

4.2. PostgreSQL and Vector Database Extension

PostgreSQL was selected as the primary database management system because of its reliability, scalability, and strong support for enterprise applications. The database stores user information, document metadata, conversation history, processing status, and other application data required by the system.

To enable semantic retrieval, PostgreSQL was extended with vector search capabilities. The vector database extension allows document embeddings to be stored and compared efficiently using similarity search algorithms. This enables the system to retrieve relevant document content based on semantic similarity rather than exact keyword matching.

4.3. Embedding Model

An embedding model was used during the document processing stage to convert document content into numerical vector representations. These embeddings capture the semantic meaning of text and make it possible to compare documents and user queries based on contextual similarity.

During processing, each uploaded document is divided into smaller text segments, and an embedding is generated for every segment before being stored in the vector database. When a user submits a

question, the same embedding process is applied to the query. The generated query embedding is then compared with the stored document embeddings to identify the most relevant document sections.

This semantic retrieval process forms the foundation of the Retrieval-Augmented Generation workflow.

4.4. Language Model

A language model was used to generate responses based on the information retrieved from the document collection. Instead of relying solely on its general knowledge, the model receives the most relevant document sections identified during the retrieval stage as contextual information before generating a response.

This approach helps ensure that generated answers remain grounded in the uploaded enterprise documents while reducing the possibility of producing unsupported or irrelevant information. It also enables users to interact with organizational knowledge using natural language, providing a more intuitive alternative to traditional document search.

4.5. Tailwind CSS

Tailwind CSS was used to design the user interface of the application. Its utility-first approach allowed responsive and consistent user interfaces to be developed efficiently while minimizing the amount of custom CSS required.

The framework was used to build the document management pages, conversational interface, navigation components, dashboards, and other interface elements, ensuring a consistent visual design throughout the application.

4.6. Blade Template Engine

Laravel's Blade template engine was used to develop the frontend views of the application. Blade supports reusable layouts, modular interface components, and template inheritance, making it easier to maintain a consistent structure across different pages.

Using Blade also reduced code duplication and helped separate presentation logic from backend functionality, improving the maintainability of the application.

4.7. JavaScript

JavaScript was used to enhance the responsiveness and interactivity of the user interface. It enabled asynchronous communication between the client and server, allowing users to interact with the application without requiring full page reloads.

The technology was also used to implement dynamic features such as live document processing updates, interactive chat functionality, notifications, modal windows, and other user interface interactions that improve the overall user experience.

4.8. Document Text Extraction

Document text extraction is one of the core components of the RAG system. A document text extraction tool was integrated into the processing pipeline to extract textual content from uploaded PDF documents while preserving the original reading order as accurately as possible.

The extracted text serves as the input for subsequent processing stages, including text preprocessing, semantic chunking, embedding generation, and indexing. Automating this process allows the system to efficiently process large collections of enterprise documents without requiring manual intervention.

4.9. Optical Character Recognition (OCR)

Some enterprise documents are available only as scanned images and do not contain searchable text. To support these documents, Optical Character Recognition (OCR) technology was integrated into the document processing workflow.

The OCR component converts scanned document images into machine-readable text, enabling these documents to undergo the same processing pipeline as digitally generated documents. This improves the usability of the system by allowing knowledge retrieval from both searchable and scanned documents.

4.10. Background Job Processing

Processing uploaded documents involves several computationally intensive operations, including text extraction, preprocessing, document chunking, embedding generation, and indexing. Executing all of these tasks immediately after upload could reduce the responsiveness of the application.

To address this, Laravel's background job processing mechanism was used to execute these operations asynchronously. This allows users to continue interacting with the application while document processing is performed in the background, resulting in a smoother and more responsive user experience.

4.11. Docker

Docker was used to containerize the application and its supporting services, providing a consistent development and deployment environment. By packaging the application together with all required dependencies, Docker minimizes environment-specific configuration issues and simplifies the setup process on different development machines.

Containerization also improves portability by ensuring that the application behaves consistently across development, testing, and deployment environments. This approach supports collaborative development and simplifies future deployment and maintenance activities.

4.12. Git and GitHub

Git was used as the version control system throughout the development of the project. It enabled changes to be tracked systematically, allowing new features, bug fixes, and improvements to be managed efficiently during development.

GitHub was used as the remote repository for source code management and collaboration. It maintained the complete development history, provided a centralized backup of the project, and supported collaborative software development by allowing changes to be reviewed, managed, and integrated in an organized manner.

4.13. Development Environment

Visual Studio Code was used as the primary integrated development environment during the project. Composer was used to manage PHP packages and project dependencies, while Node.js together with npm was used to manage frontend packages and compile application assets.

These development tools simplified dependency management, frontend asset compilation, and overall software development, contributing to a more efficient and maintainable development workflow.

5. Specification

5.1. Purpose, Scope, Users, and Constraints

Purpose

The purpose of the AI-powered Retrieval-Augmented Generation (RAG) system is to provide an intelligent document management and knowledge retrieval platform that enables users to obtain accurate information from enterprise documents through natural language interactions. Traditional document management approaches often require users to manually browse large collections of files to locate relevant information, which can be time-consuming and inefficient. The system addresses this challenge by allowing users to upload documents, organize them within a centralized repository, and ask questions in natural language to retrieve contextually relevant information.

The system is intended to improve knowledge accessibility while reducing the effort required to search through extensive technical documentation. By combining document processing, semantic retrieval, and AI-assisted response generation, the platform provides users with an efficient method for locating information without requiring prior knowledge of document structures or filenames.

Scope

The scope of the system covers the complete lifecycle of enterprise document management and intelligent knowledge retrieval. Users can upload supported document formats, organize their documents, monitor document processing, and interact with one or more processed documents through a conversational interface.

The system supports document extraction, background processing, document indexing, semantic retrieval, and AI-assisted question answering. It also provides conversation management features that allow users to maintain multiple chat sessions associated with different document collections. Throughout the document processing workflow, the system provides status tracking to inform users about the progress of each uploaded document until it becomes available for querying.

The current implementation focuses on supporting document-based knowledge retrieval within a controlled enterprise environment. Features such as enterprise authentication integration, advanced administrative analytics, and large-scale distributed deployment are considered outside the scope of the current implementation.

Intended Users

The primary users of the system are employees and knowledge workers who frequently interact with technical documents, reports, manuals, specifications, and other organizational knowledge sources. The system is designed for users who require quick access to information without manually reviewing entire documents.

In addition to general users, the system also supports administrative users responsible for managing user accounts, monitoring document processing activities, supervising system usage, and maintaining the overall operation of the platform. These administrative capabilities ensure that the system remains secure, reliable, and manageable within an enterprise environment.

Constraints

The system is subject to several technical and operational constraints that influenced its design and implementation. The quality of generated responses depends on the content extracted from uploaded documents; therefore, incomplete, low-quality, or poorly scanned documents may affect retrieval accuracy despite the availability of document processing techniques.

The current implementation supports only predefined document formats and assumes that uploaded documents are intended for organizational knowledge sharing. The system is designed for enterprise environments where document confidentiality, controlled user access, and efficient resource utilization are important considerations.

Furthermore, the platform relies on background processing for document preparation before uploaded files become available for querying. Consequently, newly uploaded documents cannot be searched until all processing stages have been completed successfully.

5.2. Functional Requirements

The functional requirements define the core capabilities that the AI-powered Retrieval-Augmented Generation (RAG) system must provide to support efficient document management and intelligent knowledge retrieval. These requirements describe the expected behaviour of the system from the user's perspective and establish the foundation for the subsequent design and implementation.

User Authentication and Access Management

The system shall provide secure user authentication to ensure that only authorized users can access the platform. Each authenticated user shall have access only to their own documents, conversations, and related resources. Administrative users shall be provided with additional privileges to manage users, monitor system activities, and perform administrative operations.

Document Management

The system shall allow users to upload, view, organize, and delete supported documents through a user-friendly interface. Uploaded documents shall be stored securely and associated with the corresponding user account. The platform shall maintain metadata for each document, including its processing status, upload date, and other relevant information.

Automated Document Processing

After a document is uploaded, the system shall automatically initiate a background processing workflow. This workflow shall extract textual content, preprocess the extracted information, divide it into manageable segments, and prepare the document for semantic retrieval. The processing workflow shall operate asynchronously to avoid interrupting the user interface.

Processing Status Monitoring

The system shall provide real-time visibility into the progress of document processing. Users shall be able to monitor each processing stage and determine whether a document is ready for querying. Appropriate status indicators shall be displayed throughout the processing lifecycle to improve transparency and user experience.

Intelligent Document Retrieval

The system shall retrieve the most relevant document content in response to user queries based on semantic similarity rather than exact keyword matching. This functionality shall enable users to locate relevant information even when different terminology is used in the uploaded documents.

AI-Assisted Question Answering

The system shall allow users to interact with processed documents using natural language questions. Based on the retrieved document content, the system shall generate context-aware responses that accurately reflect the information contained within the uploaded documents while minimizing irrelevant or unsupported answers.

Multi-Document Query Support

The platform shall support conversations across multiple selected documents. Users shall be able to choose one or more processed documents as the knowledge source for a conversation, allowing information from different documents to be considered when generating responses.

Conversation Management

The system shall provide conversation management capabilities that allow users to create, continue, rename, and review previous chat sessions. Conversation history shall be preserved to support ongoing discussions without requiring users to repeat previous queries.

Search Result References

To improve transparency and user confidence, the system shall associate generated responses with the corresponding document references whenever relevant information is retrieved. This enables users to identify the origin of the retrieved knowledge and verify the provided information.

Administrative Functions

The system shall provide administrative capabilities for monitoring overall system usage and maintaining platform operations. Administrative users shall be able to supervise document processing activities, manage user accounts, monitor background jobs, and perform maintenance tasks necessary for reliable system operation.

5.3. Non-functional Requirements

In addition to its functional capabilities, the AI-powered Retrieval-Augmented Generation (RAG) system must satisfy several non-functional requirements to ensure reliable, secure, and efficient operation within an enterprise environment. These requirements define the quality attributes that influence the overall usability, maintainability, and performance of the system.

Performance

The system shall provide responsive interactions for both document management and conversational querying. Document processing tasks shall be executed asynchronously in the background to prevent blocking user interactions during upload and preparation. Once a document has been processed successfully, query responses should be generated within an acceptable response time to provide a smooth user experience.

Scalability

The system shall be designed to accommodate increasing numbers of users, documents, and conversations without requiring significant architectural changes. The document processing workflow and retrieval mechanism should support future expansion, allowing additional computing resources or storage capacity to be incorporated as system demand grows.

Security

The platform shall ensure that user documents and conversations remain protected from unauthorized access. User authentication and authorization mechanisms shall restrict access to resources based on user roles. Documents shall remain isolated between users, ensuring that only authorized individuals can access their own uploaded content. Administrative operations shall be available only to users with appropriate privileges.

Reliability

The system shall provide reliable document processing and retrieval services under normal operating conditions. Background processing tasks should execute independently of user interactions, reducing the risk of interrupted operations. Appropriate error handling and status reporting shall be implemented to ensure that failures are detected, reported, and managed without affecting overall system stability.

Availability

The platform shall remain accessible during normal operating hours with minimal service interruptions. Long-running document processing operations shall not prevent users from accessing other system features, such as viewing previously processed documents or continuing existing conversations.

Usability

The user interface shall provide an intuitive and consistent experience for users with varying levels of technical expertise. Common tasks such as uploading documents, monitoring processing status, selecting knowledge sources, and interacting with the conversational interface shall require minimal user effort. Clear navigation and informative status indicators shall improve user understanding throughout the document processing lifecycle.

Maintainability

The system shall be developed using a modular software architecture to simplify future maintenance and feature enhancements. Individual components responsible for document processing, retrieval, conversation management, and administration shall remain loosely coupled, enabling modifications or extensions without significantly affecting other parts of the system. Clear project organization and documentation shall further support long-term maintainability.

Compatibility

The system shall support commonly used enterprise document formats and operate through modern web browsers without requiring additional client-side software. The application shall provide a consistent user experience across different desktop operating systems while maintaining compatibility with the underlying enterprise infrastructure.

Extensibility

The architecture shall allow new document formats, retrieval techniques, administrative features, and AI capabilities to be integrated with minimal changes to the existing system. This extensible design ensures that the platform can evolve alongside future business requirements and technological advancements.

5.4. User Stories

User Story 1: User Authentication

As a user,

I want to securely log in to the system,

So that I can access my personal workspace and interact with my documents.

Acceptance Criteria:

- The system authenticates users before granting access.
- Only registered users can access the platform.
- Users can access only their own documents and conversations.
- Invalid login attempts result in an appropriate error message.

User Story 2: Uploading Documents

As a user,

I want to upload documents to the system,

So that they can be processed and used as knowledge sources for future conversations.

Acceptance Criteria:

- The system allows supported document formats to be uploaded.
- Uploaded documents are stored securely and associated with the authenticated user.
- A document processing workflow starts automatically after a successful upload.
- The user receives confirmation that the upload has been completed successfully.

User Story 3: Monitoring Document Processing

As a user,

I want to monitor the processing status of my uploaded documents,

So that I know when they are ready to be queried.

Acceptance Criteria:

- The system displays the current processing status of each uploaded document.
- Processing stages are updated automatically as background tasks progress.
- The document is marked as ready once processing has been completed successfully.
- Appropriate error messages are displayed if processing fails.

User Story 4: Managing Documents

As a user,

I want to view and manage my uploaded documents,

So that I can organize my knowledge base efficiently.

Acceptance Criteria:

- The system displays a list of uploaded documents.
- Users can view document details and processing information.
- Users can remove documents that are no longer required.
- Changes are reflected immediately after the requested operation.

User Story 5: Starting a Conversation

As a user,

I want to start a new conversation by selecting one or more processed documents,

So that I can retrieve information from the relevant knowledge sources.

Acceptance Criteria:

- The system allows users to select one or multiple processed documents.
- A new conversation is created successfully.
- Only documents that have completed processing can be selected.
- The selected documents become the knowledge source for the conversation.

User Story 6: Asking Questions

As a user,

I want to ask questions in natural language,

So that I can quickly obtain relevant information without manually searching through documents.

Acceptance Criteria:

- The system accepts natural language queries.
- Questions are processed using the selected documents.
- Responses are generated based on the retrieved document content.
- If no relevant information is available, the system informs the user appropriately.

User Story 7: Retrieving Information from Multiple Documents

As a user,

I want to retrieve information from multiple documents within a single conversation,

So that I can obtain comprehensive answers from different knowledge sources.

Acceptance Criteria:

- Multiple processed documents can be selected for a conversation.
- The system retrieves relevant information from all selected documents.
- Responses are generated using the combined retrieved context.
- Only the selected documents are considered during retrieval.

User Story 8: Viewing Response References

As a user,

I want to view the document references associated with generated responses,

So that I can verify the source of the retrieved information.

Acceptance Criteria:

- Generated responses include references to the corresponding document sources.
- References clearly indicate the originating document.
- Users can identify where the retrieved information was obtained.
- References are displayed together with the generated response.

User Story 9: Managing Conversation History

As a user,

I want to access my previous conversations,

So that I can continue earlier discussions without losing context.

Acceptance Criteria:

- The system stores previous conversations.
- Users can view a list of existing conversations.
- Selecting a conversation restores its previous messages.
- Users can continue asking questions within an existing conversation.

User Story 10: Administering the System

As an administrator,

I want to manage users and monitor system activities,

So that the platform remains secure, reliable, and properly maintained.

Acceptance Criteria:

- Administrative users have access to system management functions.
- Administrators can monitor document processing activities.
- Administrators can manage user accounts and system resources.
- Administrative features are accessible only to authorized users.

5.5. Use Case Diagrams and Use Case Descriptions

The use case diagram represents the main interactions between users and the AI-powered document retrieval system. It shows how general users and administrators interact with the system to upload documents, monitor processing, manage conversations, ask questions, and maintain system operations.

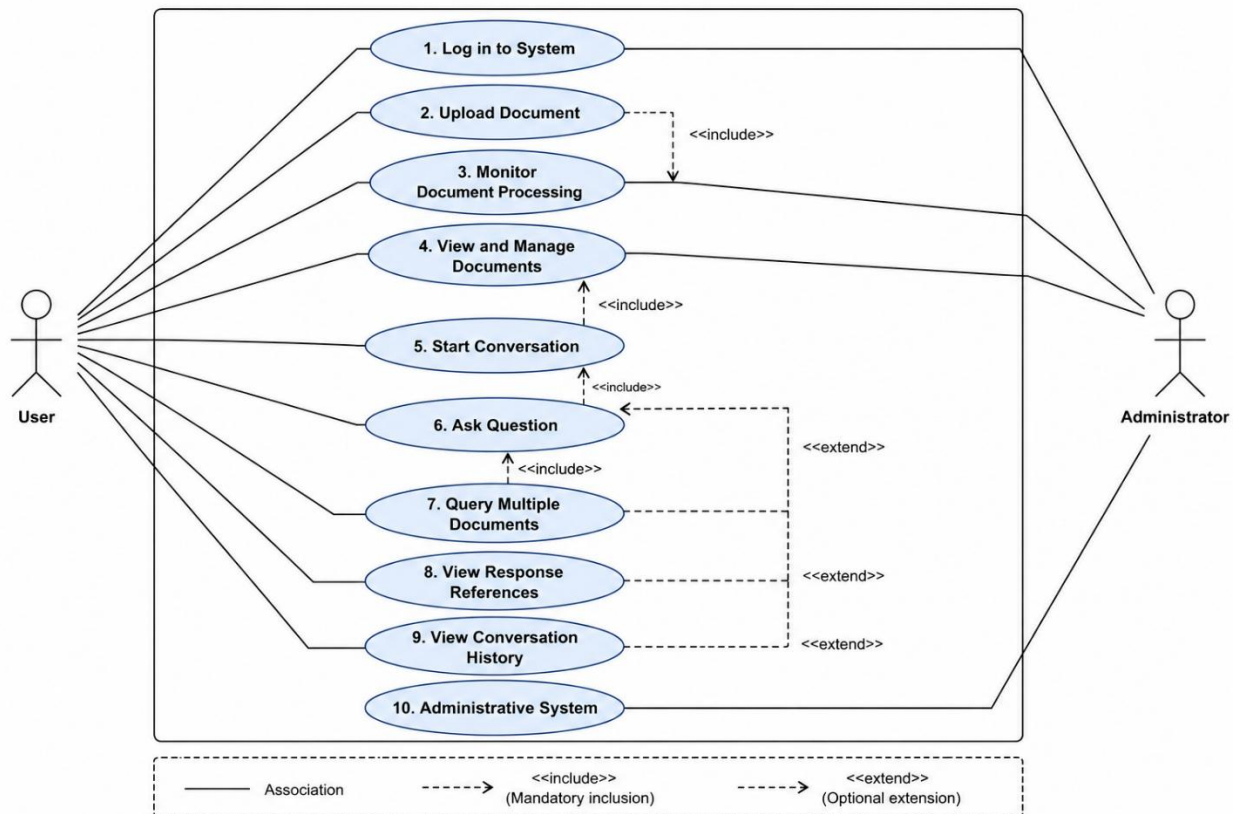


Figure 5.1: Overall Use Case Diagram of the AI-powered Document Retrieval System

The diagram illustrates the main system functions available to users and administrators. A regular user can log in, upload documents, monitor document processing, manage uploaded documents, start conversations, ask questions, query multiple documents, view response references, and access previous conversation history. The administrator has additional responsibilities related to system supervision, user management, and monitoring document processing activities.

In the diagram, “Query Multiple Documents” is represented as an extension of “Ask Question” because it is a specialized form of the normal question-answering process where more than one document is selected as the knowledge source. Similarly, “View Response References” is represented as an extension

of “Ask Question” because references are displayed after a response is generated and allow the user to verify the source of the retrieved information. “View Conversation History” is kept as an independent use case because users may review previous conversations separately from asking a new question.

Use Case 1: Log in to System

Actors: User, Administrator

Description: This use case describes how a user or administrator logs in to the system to access the platform securely.

Preconditions:

- The user must have a registered account.
- The system must be available.

Postconditions:

- The user is authenticated successfully.
- The user is redirected to the appropriate dashboard.

Normal Flow:

1. The user opens the login page.
2. The user enters valid login credentials.
3. The system validates the credentials.
4. The system grants access to the user.
5. The user is redirected to the main dashboard.

Alternative Flows and Exceptions:

- If the credentials are invalid, the system displays an error message.
- If the user account is inactive, the system denies access.
- If the system is unavailable, the user cannot proceed with login.

Use Case 2: Upload Document

Actors: User

Description: This use case describes how a user uploads a supported document to the system so that it can be processed for future retrieval.

Preconditions:

- The user must be logged in.
- The uploaded file must be in a supported document format.

Postconditions:

- The document is uploaded successfully.
- The document processing workflow is started.

Normal Flow:

1. The user navigates to the document upload page.

2. The user selects a supported document from the local device.
3. The user submits the upload form.
4. The system validates the file type and file size.
5. The system stores the document and associates it with the user account.
6. The system starts the document processing workflow.
7. The system displays a confirmation message.

Alternative Flows and Exceptions:

- If the file format is not supported, the system rejects the upload.
- If the file size exceeds the allowed limit, the system displays an error message.
- If the upload fails, the system informs the user and keeps the document unprocessed.

Use Case 3: Monitor Document Processing

Actors: User, Administrator

Description: This use case describes how users monitor the processing status of uploaded documents until they are ready for querying.

Preconditions:

- The user must be logged in.
- At least one document must have been uploaded.

Postconditions:

- The user can see the current processing status of the document.
- The document is marked as ready when processing is completed.

Normal Flow:

1. The user opens the document list.
2. The system displays the uploaded documents.
3. The system shows the current processing status for each document.
4. The status is updated as the processing workflow progresses.
5. When processing is completed, the document is marked as ready.

Alternative Flows and Exceptions:

- If processing fails, the system displays a failure status.
- If the document is still being processed, the system prevents it from being used in a conversation.
- If the status cannot be retrieved, the system displays the last known status.

Use Case 4: View and Manage Documents

Actors: User, Administrator

Description: This use case describes how users view, organize, and manage uploaded documents within their workspace.

Preconditions:

- The user must be logged in.
- The user must have uploaded documents.

Postconditions:

- The user can view document information.
- The user can remove documents that are no longer required.

Normal Flow:

1. The user opens the document management page.
2. The system displays the list of uploaded documents.
3. The user views document details such as name, upload date, and processing status.
4. The user performs a management action, such as viewing details or deleting a document.
5. The system updates the document list accordingly.

Alternative Flows and Exceptions:

- If no documents are available, the system displays an empty state message.
- If the user deletes a document, the system asks for confirmation before removal.
- If deletion fails, the system displays an error message.

Use Case 5: Start Conversation

Actors: User

Description: This use case describes how a user starts a new conversation by selecting one or more processed documents as the knowledge source.

Preconditions:

- The user must be logged in.
- At least one document must be processed and ready for querying.

Postconditions:

- A new conversation is created.
- The selected documents are linked to the conversation.

Normal Flow:

1. The user opens the conversation interface.
2. The user selects one or more processed documents.
3. The user creates a new conversation.
4. The system stores the conversation details.
5. The system opens the chat interface for the selected documents.

Alternative Flows and Exceptions:

- If no processed documents are available, the system prevents conversation creation.
- If the user selects an unprocessed document, the system displays a warning.
- If conversation creation fails, the system displays an error message.

Use Case 6: Ask Question

Actors: User

Description: This use case describes how a user asks a natural language question and receives an answer based on the selected documents.

Preconditions:

- The user must be logged in.
- A conversation must exist.
- At least one processed document must be selected.

Postconditions:

- The user question is stored in the conversation.
- The system generates and displays a response.

Normal Flow:

1. The user opens an active conversation.
2. The user enters a question in natural language.
3. The system searches the selected document content.
4. The system retrieves relevant information.
5. The system generates a response based on the retrieved content.
6. The response is displayed to the user.

Alternative Flows and Exceptions:

- If no relevant information is found, the system informs the user.
- If response generation fails, the system displays an error message.
- If the selected document is no longer available, the system prevents the query.

Use Case 7: Query Multiple Documents

Actors: User

Description: This use case describes how a user retrieves information from multiple selected documents within a single conversation.

Preconditions:

- The user must be logged in.
- Multiple processed documents must be available.
- The user must have selected more than one document.

Postconditions:

- The system retrieves relevant information from the selected documents.
- The response is generated using the combined retrieved context.

Normal Flow:

1. The user selects multiple processed documents.
2. The user starts or opens a conversation.

3. The user asks a question.
4. The system searches across all selected documents.
5. The system retrieves the most relevant content.
6. The system generates a response using the retrieved information.

Alternative Flows and Exceptions:

- If one of the selected documents is unavailable, the system continues with the available documents.
- If no relevant content is found, the system displays an appropriate message.
- If too many documents are selected, the system may limit retrieval based on system constraints.

Use Case 8: View Response References

Actors: User

Description: This use case describes how users view document references related to generated responses.

Preconditions:

- The user must be logged in.
- A response must have been generated.
- Relevant document content must have been retrieved.

Postconditions:

- The user can view the document references associated with the response.

Normal Flow:

1. The user asks a question.
2. The system retrieves relevant document content.
3. The system generates a response.
4. The system displays the response with related document references.
5. The user reviews the references to verify the source of the information.

Alternative Flows and Exceptions:

- If no references are available, the system displays the response without references.
- If the source document has been deleted, the reference may no longer be accessible.
- If retrieval fails, no reference is displayed.

Use Case 9: View Conversation History

Actors: User

Description: This use case describes how users view previous conversations and continue earlier discussions.

Preconditions:

- The user must be logged in.
- At least one conversation must exist.

Postconditions:

- The selected conversation is displayed.
- The user can review and continue the conversation.

Normal Flow:

1. The user opens the conversation history panel.
2. The system displays the list of previous conversations.
3. The user selects a conversation.
4. The system loads the previous messages.
5. The user reviews the conversation or continues asking questions.

Alternative Flows and Exceptions:

- If no previous conversations exist, the system displays an empty state message.
- If the selected conversation cannot be loaded, the system displays an error.
- If related documents are unavailable, the system informs the user before continuing.

Use Case 10: Administrative System

Actors: Administrator

Description: This use case describes how an administrator monitors and manages system activities to ensure stable and secure operation.

Preconditions:

- The administrator must be logged in.
- The administrator must have appropriate privileges.

Postconditions:

- System activities are monitored.
- Required administrative actions are completed.

Normal Flow:

1. The administrator logs in to the system.
2. The administrator opens the administration dashboard.
3. The system displays user, document, and processing information.
4. The administrator reviews system activity.
5. The administrator performs required management actions.
6. The system applies the administrative changes.

Alternative Flows and Exceptions:

- If the user does not have administrator privileges, access is denied.
- If system activity data cannot be loaded, the system displays an error message.
- If an administrative action fails, the system logs the error and informs the administrator.

5.6. External Interfaces

The AI-powered document retrieval system interacts with several external components that support document processing, semantic retrieval, response generation, data storage, and application deployment. These interfaces enable the system to communicate with external services while keeping the core application independent of their internal implementations. The following subsections describe the primary external interfaces used throughout the system.

5.6.1. Database Interface

The system uses a PostgreSQL database with vector storage capabilities as its primary persistence layer. The database stores user information, uploaded document metadata, processed document chunks, conversation history, and vector embeddings generated during document processing. By separating structured application data from the application logic, the database provides efficient storage, retrieval, and indexing of information required by the retrieval pipeline.

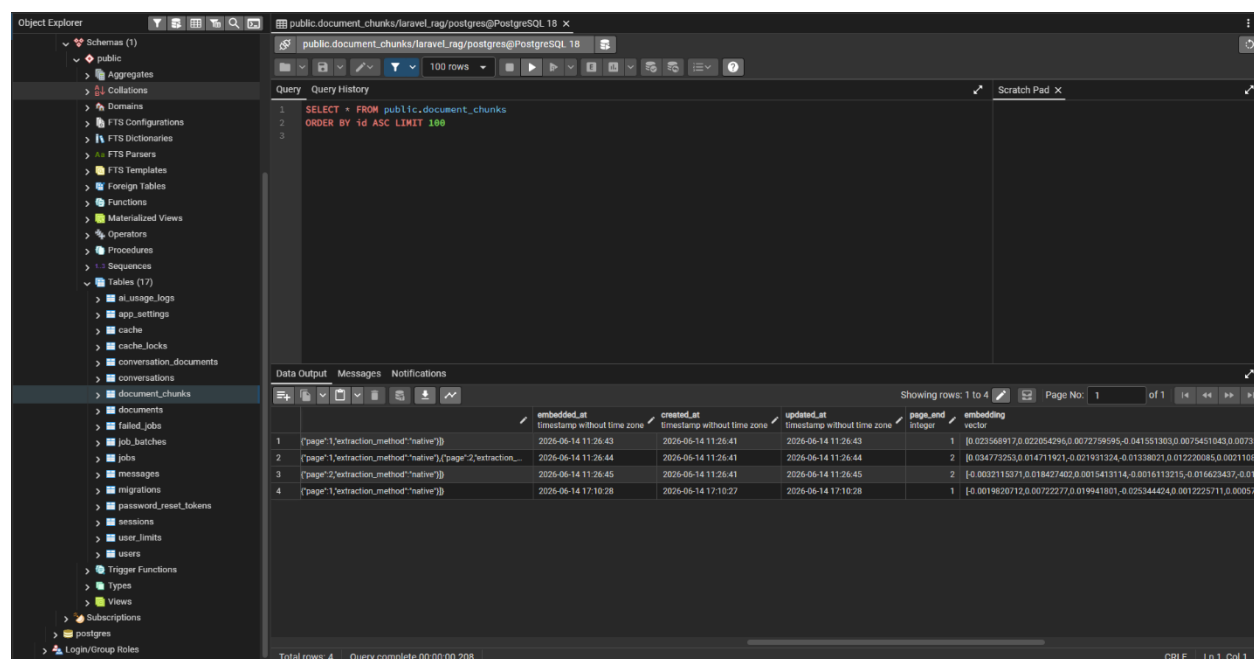


Figure 5.2: PostgreSQL Database Interface

5.6.2. Embedding Model Interface

The embedding model interface is responsible for transforming processed document chunks into high-dimensional vector representations. These embeddings capture the semantic meaning of the document content and enable similarity-based retrieval during user queries. The application communicates with the embedding service during the document processing pipeline, and the generated vectors are subsequently stored in the database for future retrieval operations.

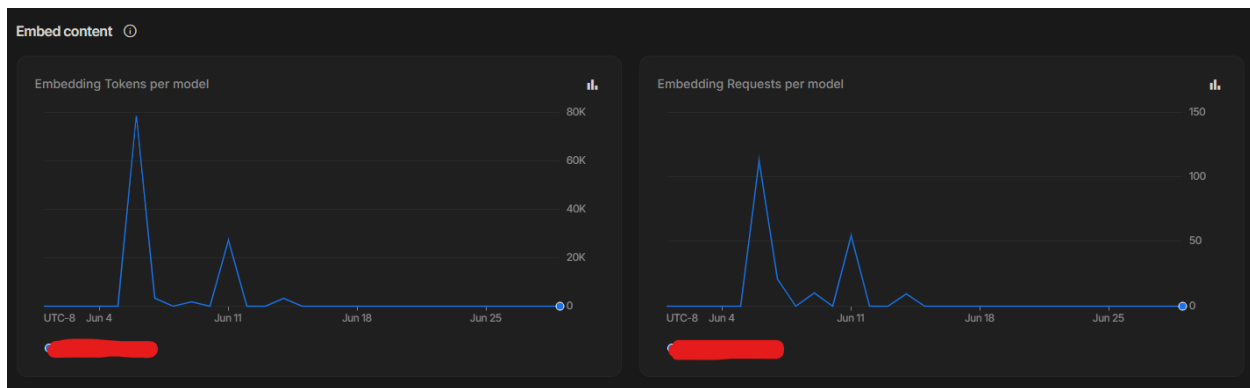


Figure 5.3: Embedding Model Interface Used for Document Vector Generation

5.6.3. Language Model Interface

The language model interface is used during the question-answering process. After relevant document content has been retrieved, the selected context together with the user's question is forwarded to the language model, which generates a context-aware response. The application exchanges prompts and responses through this interface while ensuring that the generated answer is based on the retrieved document context.

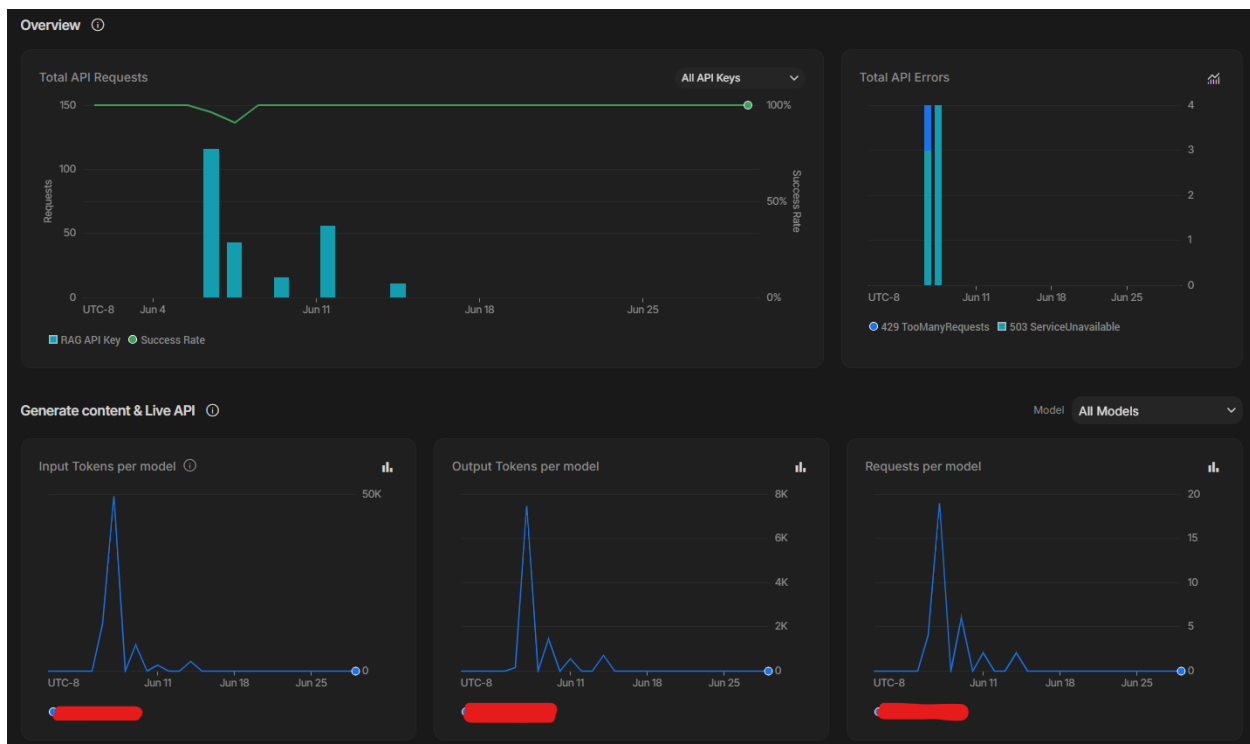
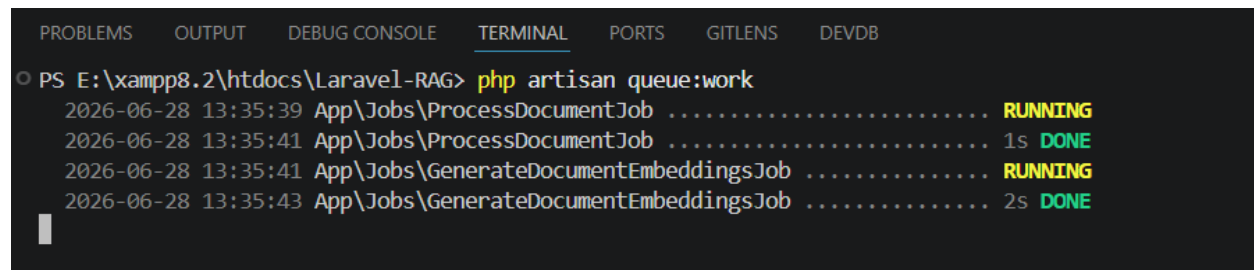


Figure 5.4: Language Model Interface Used for Context-Aware Response Generation

5.6.4. Background Processing Interface

Document processing is performed asynchronously through a background job processing mechanism. After a document is uploaded, the system automatically schedules a series of processing jobs responsible for text extraction, document chunking, embedding generation, and database storage. Executing these

operations in the background prevents long-running tasks from blocking user interactions and allows users to monitor processing progress while the application continues operating normally.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS DEVDB
PS E:\xampp8.2\htdocs\Laravel-RAG> php artisan queue:work
2026-06-28 13:35:39 App\Jobs\ProcessDocumentJob ..... RUNNING
2026-06-28 13:35:41 App\Jobs\ProcessDocumentJob ..... 1s DONE
2026-06-28 13:35:41 App\Jobs\GenerateDocumentEmbeddingsJob ..... RUNNING
2026-06-28 13:35:43 App\Jobs\GenerateDocumentEmbeddingsJob ..... 2s DONE
```

Figure 5.5: Background Job Processing Interface

5.6.5. Containerization Interface

To simplify development, deployment, and environment consistency, the application is containerized using Docker. Individual containers are used to isolate the web application, database, web server, and background processing services while allowing them to communicate through a shared container network. This containerized architecture ensures that the application can be deployed consistently across different development and production environments with minimal configuration changes.

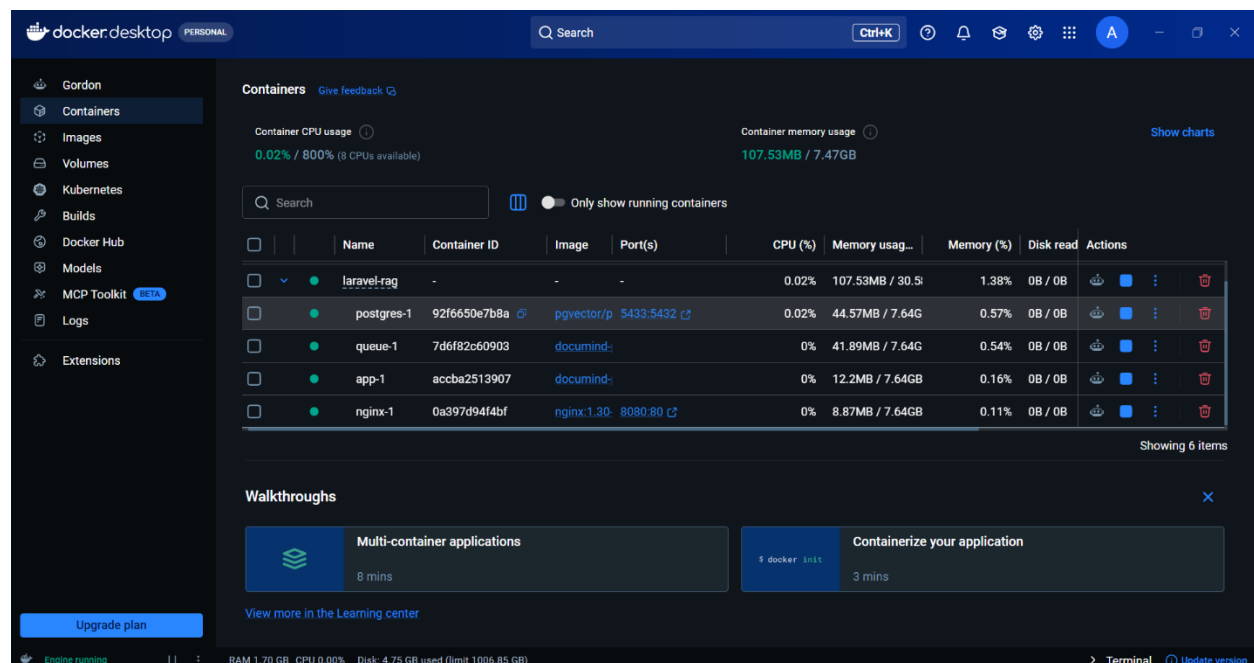


Figure 5.6: Docker Container Environment

5.7. User Interface Mockups

The user interface of the AI-powered document retrieval system was designed to provide a simple and intuitive experience for both end users and administrators. The application follows a dashboard-based layout that enables users to manage documents, monitor processing progress, and interact with the conversational retrieval system efficiently. Each interface was developed with a focus on usability,

consistency, and ease of navigation, ensuring that users can perform their tasks with minimal complexity. This section presents the primary interfaces implemented during the development of the system.

5.7.1. Login Interface

The login interface provides secure authentication before users can access the application. Only registered users with valid credentials are permitted to enter the workspace. Upon successful authentication, users are redirected to their respective dashboards according to their assigned roles. This interface serves as the primary security mechanism for protecting uploaded documents and conversation data.

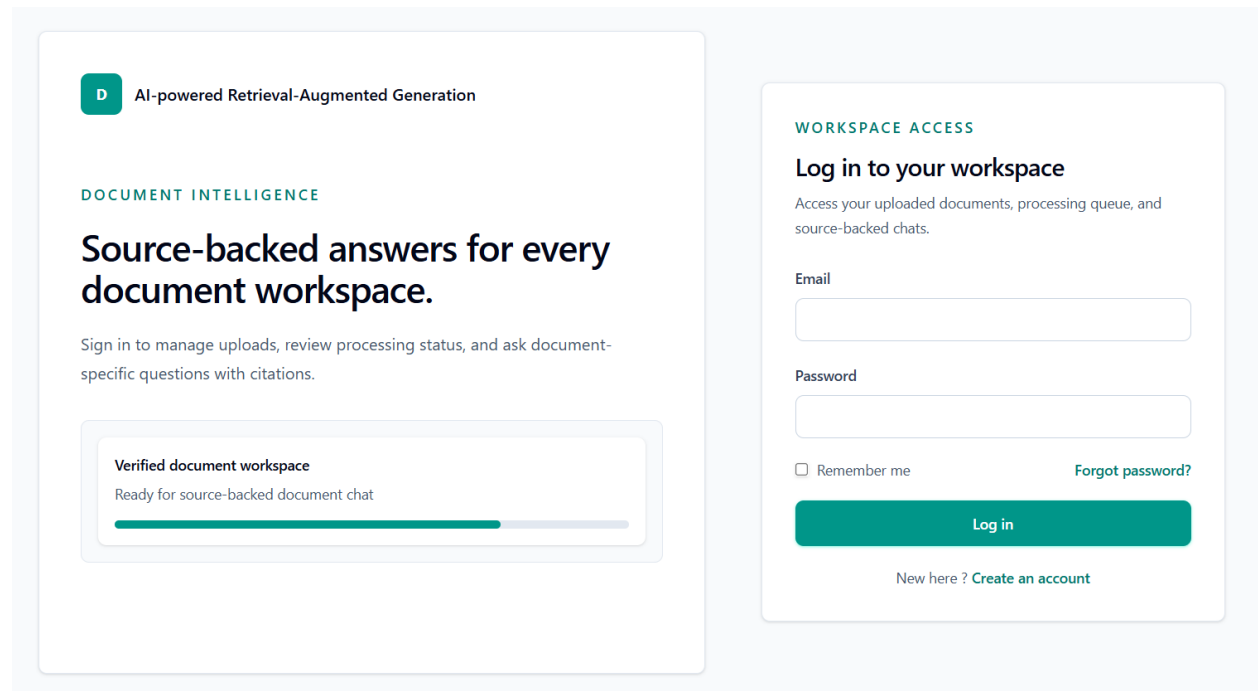


Figure 5.7: Login Interface

5.7.2. User Dashboard

The user dashboard serves as the central workspace after successful authentication. It provides an overview of the user's document repository by displaying key statistics such as the total number of uploaded documents, documents ready for querying, documents currently being processed, and the total number of questions asked. The dashboard also presents recently uploaded documents, enabling users to quickly monitor document availability and access frequently used resources.

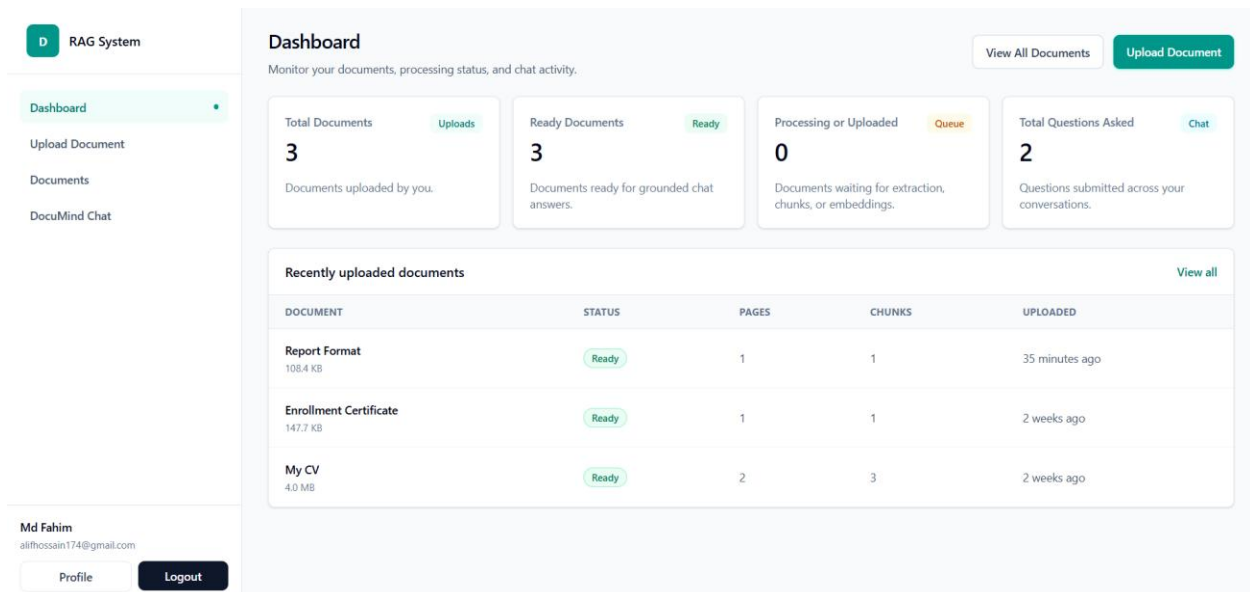


Figure 5.8: User Dashboard

5.7.3. Document Upload Interface

The document upload interface enables users to submit supported document formats for processing. Users may optionally provide document metadata, such as a title and description, before uploading the file. After submission, the application automatically initiates the background processing workflow, including text extraction, document chunking, and embedding generation. The interface also displays recently uploaded documents together with their current processing status.

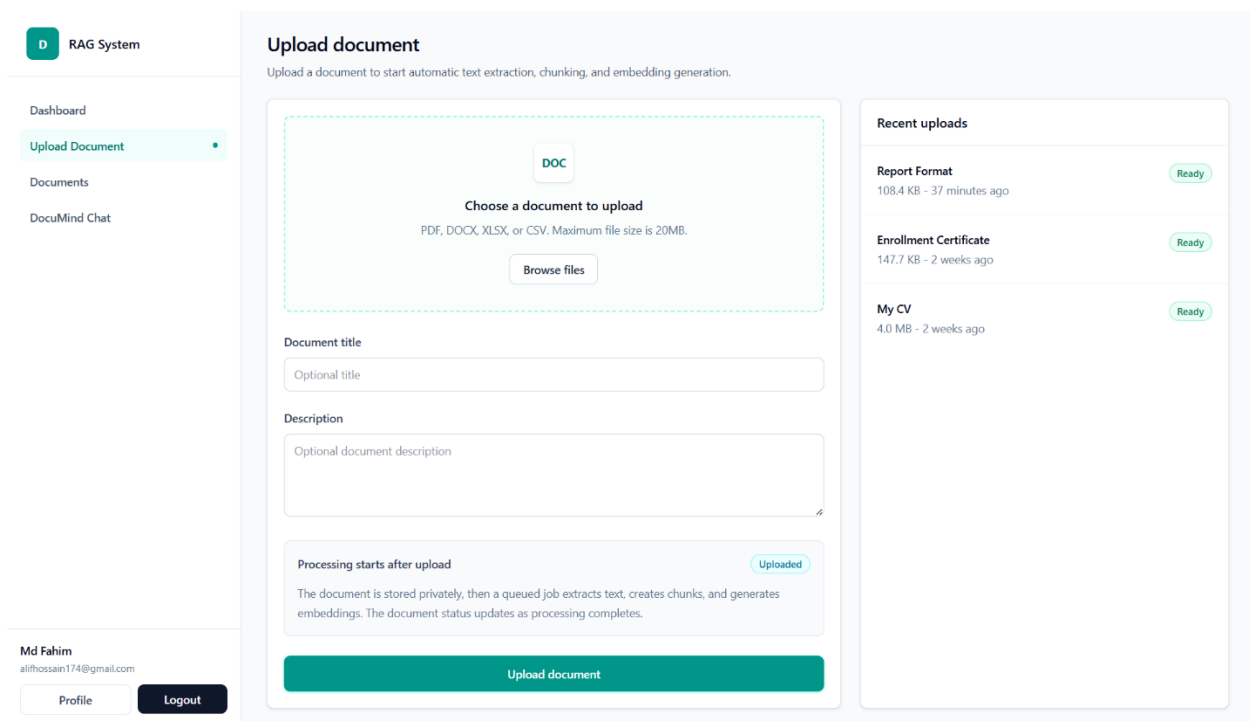


Figure 5.9: Document Upload Interface

5.7.4. Document Details and Processing Status

The document details interface provides comprehensive information about an uploaded document together with its processing timeline. Users can monitor each stage of the document processing pipeline, including upload completion, text extraction, chunk creation, embedding generation, and readiness for conversational querying. The interface additionally displays document metadata and previews of the generated document chunks, allowing users to verify the extracted content before initiating conversations with the system.

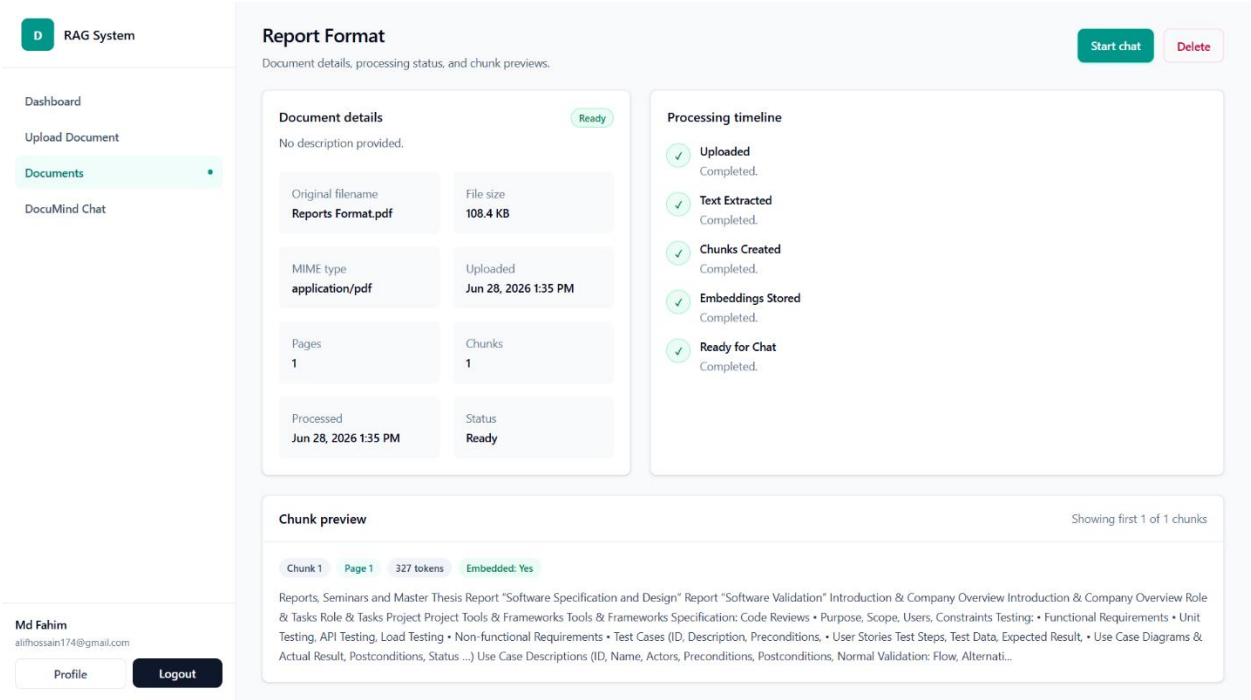


Figure 5.10: Document Details and Processing Status Interface

5.7.5. Conversation Interface

The conversation interface represents the primary functionality of the AI-powered document retrieval system. Users can create new conversations by selecting one or multiple processed documents and subsequently ask natural language questions based on the selected knowledge sources. The interface displays AI-generated responses together with document citations, suggested follow-up questions, and conversation controls, providing an intuitive and interactive environment for document-based knowledge retrieval.

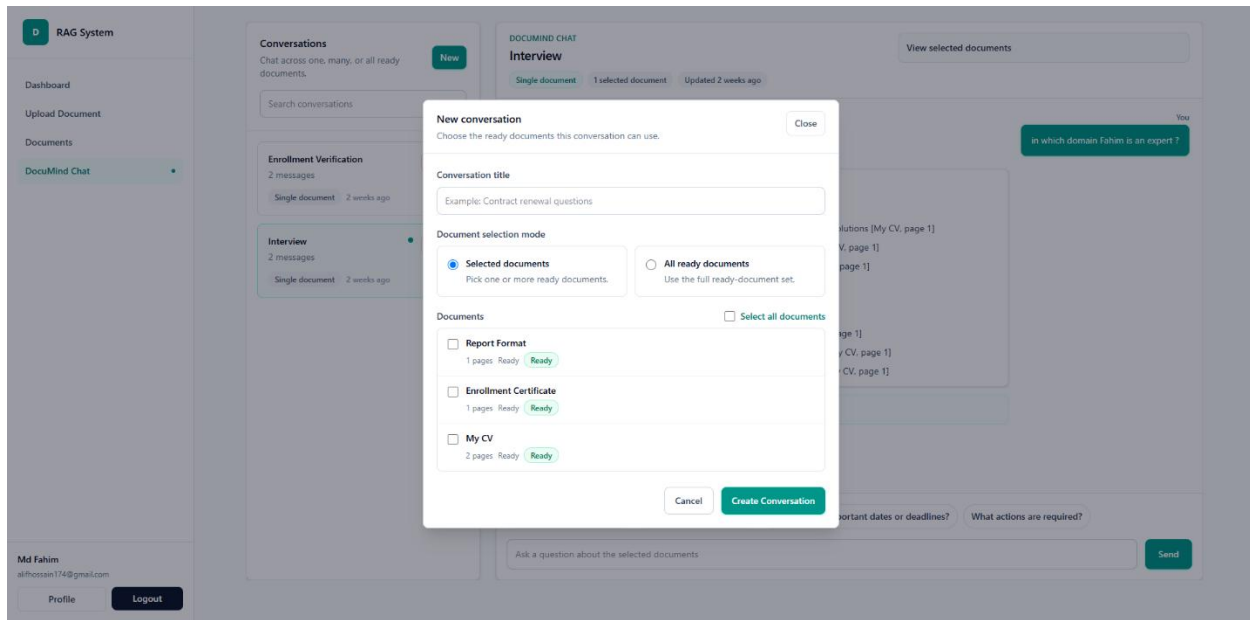


Figure 5.11: Conversation Interface

5.7.6. Conversation History

The conversation history interface enables users to revisit previous discussions and continue existing conversations without losing context. Conversations are presented in a searchable list together with relevant metadata, including the document scope and the last interaction time. This functionality supports long-term knowledge management by allowing users to review previously generated responses and continue earlier conversations whenever required.

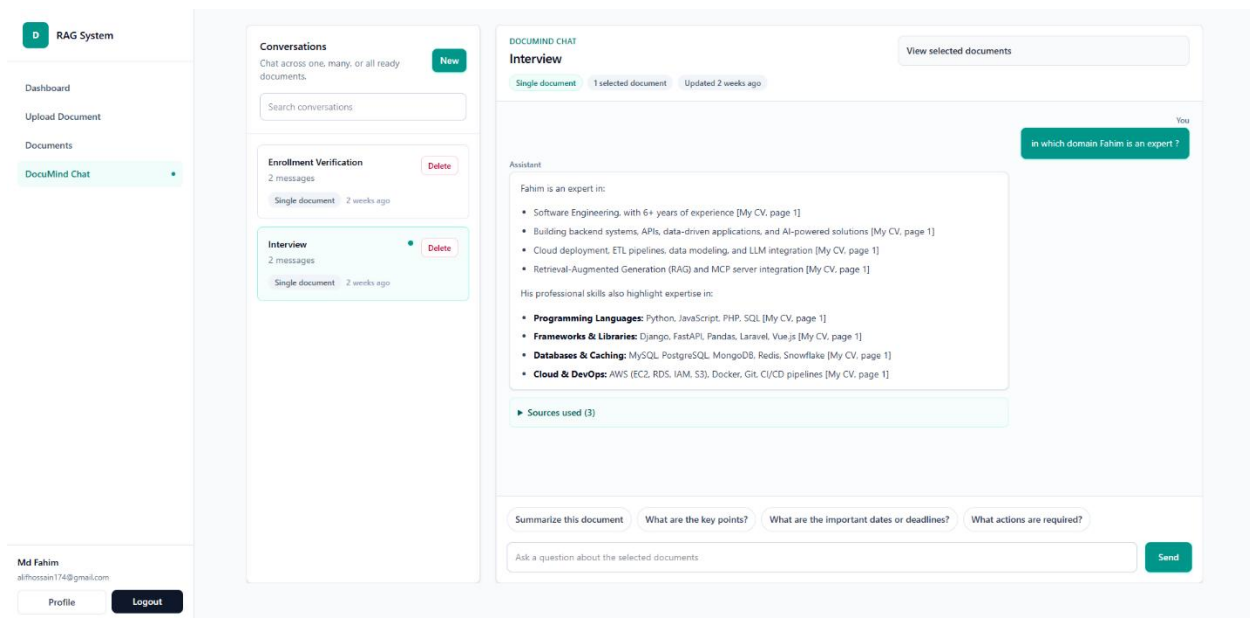


Figure 5.12: Conversation History Interface

5.7.7. Administrative Dashboard

The administrative dashboard provides centralized management and monitoring capabilities for the overall system. It presents operational statistics related to users, uploaded documents, document processing, and generated document chunks through a single interface. Administrators can also access dedicated modules for user management, document supervision, usage limits, background queue monitoring, failed job management, and system configuration. By consolidating these administrative functions into a unified dashboard, the interface supports efficient system supervision and helps maintain the reliable operation of the AI-powered document retrieval system.

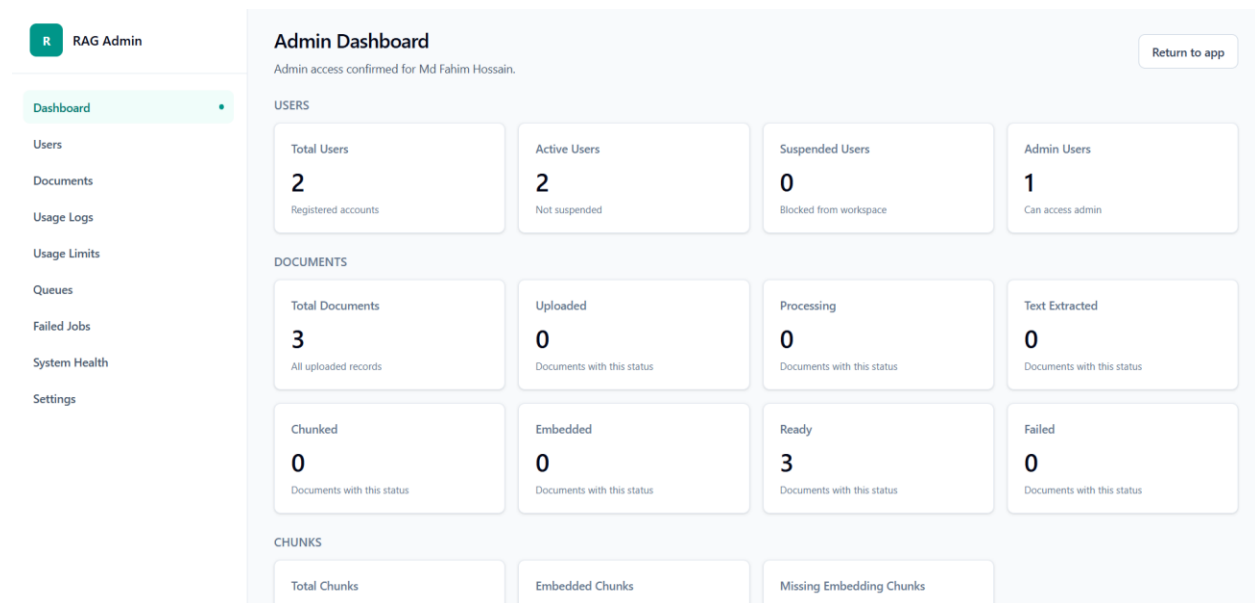


Figure 5.13: Administrative Dashboard

6. Design

6.1. System Architecture

The architecture of the AI-powered document retrieval system was designed with the objectives of modularity, scalability, and maintainability. Since the application combines traditional web application features with artificial intelligence capabilities, the architecture separates the responsibilities of user interaction, business logic, document processing, knowledge retrieval, and data storage into independent layers. This separation allows each component to evolve independently while maintaining a clear and well-defined communication flow throughout the system.

The system follows a layered architecture in which users interact with the application through a web-based interface developed using Laravel Blade templates and Tailwind CSS. User requests are processed by the Laravel application, which acts as the central controller responsible for authentication, document management, conversation management, and coordination of the document processing pipeline. This approach keeps the presentation layer independent from the underlying processing logic, resulting in a clean and maintainable system structure.

A dedicated document processing pipeline is responsible for transforming uploaded documents into searchable knowledge. When a user uploads a supported document, the system stores the original file and initiates a series of background processing tasks. These tasks include text extraction, optional optical character recognition for scanned documents, text cleaning, document chunking, embedding generation, and storage of vector representations within the database. Executing these operations asynchronously prevents long waiting times for users while ensuring that computationally intensive tasks do not affect the responsiveness of the application.

The knowledge retrieval process is separated from document processing and is only activated when a user submits a question. Instead of searching entire documents directly, the system generates an embedding for the user's query and performs a semantic similarity search against the stored document embeddings. Only the most relevant document chunks are retrieved and combined with the user's question before being forwarded to the language model. This Retrieval-Augmented Generation (RAG) approach improves answer relevance while reducing unnecessary processing and limiting the amount of document content sent during inference.

The persistence layer consists of a PostgreSQL database enhanced with vector storage capabilities. Besides managing traditional relational data such as users, conversations, uploaded documents, and processing status, the database also stores vector embeddings for semantic search. This unified storage approach simplifies data management while providing efficient retrieval of relevant document content during question answering.

Several external services are integrated into the architecture to provide specialized capabilities. Text extraction libraries process uploaded documents, OCR tools handle scanned files when necessary, embedding models convert text into numerical vector representations, and a language model generates context-aware responses. These services are accessed through dedicated service classes, ensuring loose coupling between the application core and external dependencies. Such abstraction improves maintainability and allows individual services to be replaced in the future with minimal impact on the overall system.

Overall, the architecture provides a clear separation between user interaction, document processing, semantic retrieval, artificial intelligence services, and data persistence. This modular design improves maintainability, supports future extensions, and enables the system to efficiently process enterprise documents while delivering context-aware responses through the Retrieval-Augmented Generation workflow.

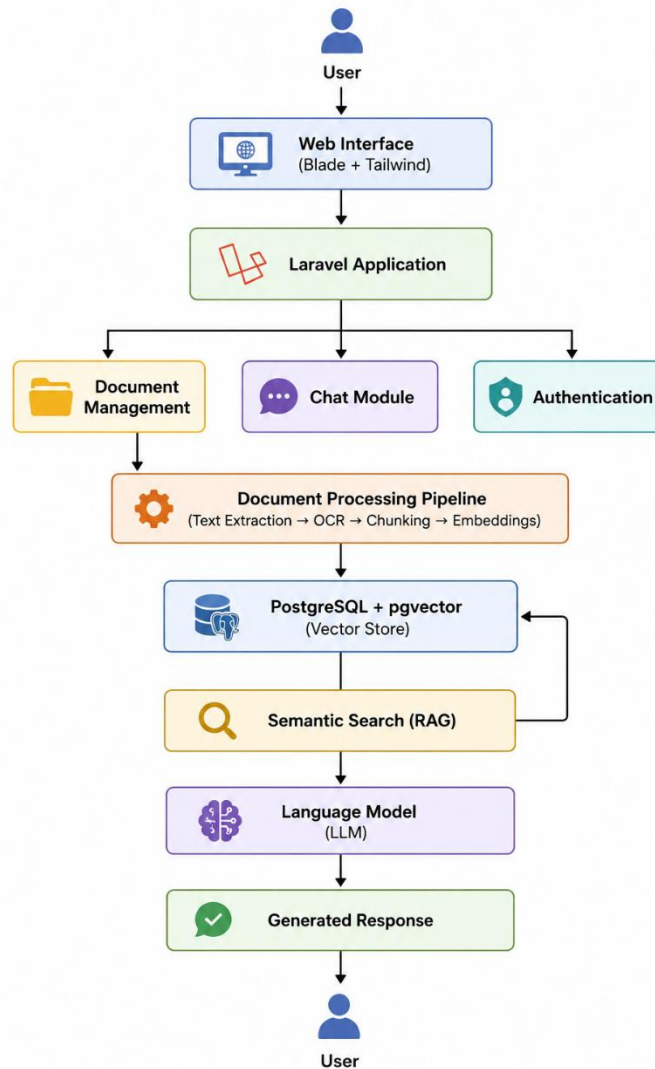


Figure 6.1: High-Level Architecture of the AI-Powered Enterprise RAG System

6.2. Component Diagram

The component diagram provides a structural view of the major software modules that make up the AI-powered Retrieval-Augmented Generation (RAG) system. While the system architecture presented in the previous section illustrates the overall workflow of the application, the component diagram focuses on the internal organization of the software and the relationships between its primary functional modules.

The system follows a modular design in which each component is responsible for a specific set of functionalities. This separation of responsibilities reduces coupling between different parts of the application and improves maintainability, scalability, and extensibility. Individual components communicate through well-defined interfaces while remaining independent of each other's internal implementations.

At the center of the application is the Laravel Application Core, which coordinates communication between all other components. It manages incoming user requests, executes business logic, invokes document processing services, performs semantic retrieval, and communicates with external artificial intelligence services.

The User Interface component provides the web-based frontend through which users upload documents, manage conversations, and interact with the AI assistant. User requests are forwarded to the application core, while generated responses and document information are returned to the interface for presentation.

The Authentication component manages user authentication and authorization. It verifies user identities, enforces access control, and ensures that users can access only their own documents and conversations, while administrators are granted additional management capabilities.

The Document Management component is responsible for handling document uploads, storage, metadata management, and document lifecycle operations. Once a document is uploaded successfully, this component initiates the document processing workflow.

The Document Processing component performs all operations required to prepare uploaded documents for semantic retrieval. These operations include text extraction, optical character recognition when necessary, text preprocessing, document chunking, and embedding generation. Processing tasks are executed asynchronously through the background job system to improve application responsiveness.

The Semantic Retrieval component performs similarity search against the stored document embeddings. After converting the user's question into an embedding, it retrieves the most relevant document chunks from the vector database and prepares the contextual information required for response generation.

The Conversation Management component maintains conversation history, stores user messages and AI responses, manages document selections for each conversation, and preserves conversational context across multiple interactions.

The AI Integration component provides communication with external artificial intelligence services responsible for embedding generation and context-aware response generation. By isolating these interactions within a dedicated component, the application remains independent of specific AI service implementations, allowing future replacement or extension with minimal architectural changes.

Finally, the Database component serves as the persistent storage layer for the application. It stores user accounts, uploaded documents, document chunks, vector embeddings, conversations, processing status, and other application data required for system operation.

Overall, the component-based architecture enables the application to separate user interaction, document processing, semantic retrieval, conversation management, and persistent storage into independent modules. This modular organization simplifies maintenance, improves scalability, and provides a solid foundation for future system enhancements.

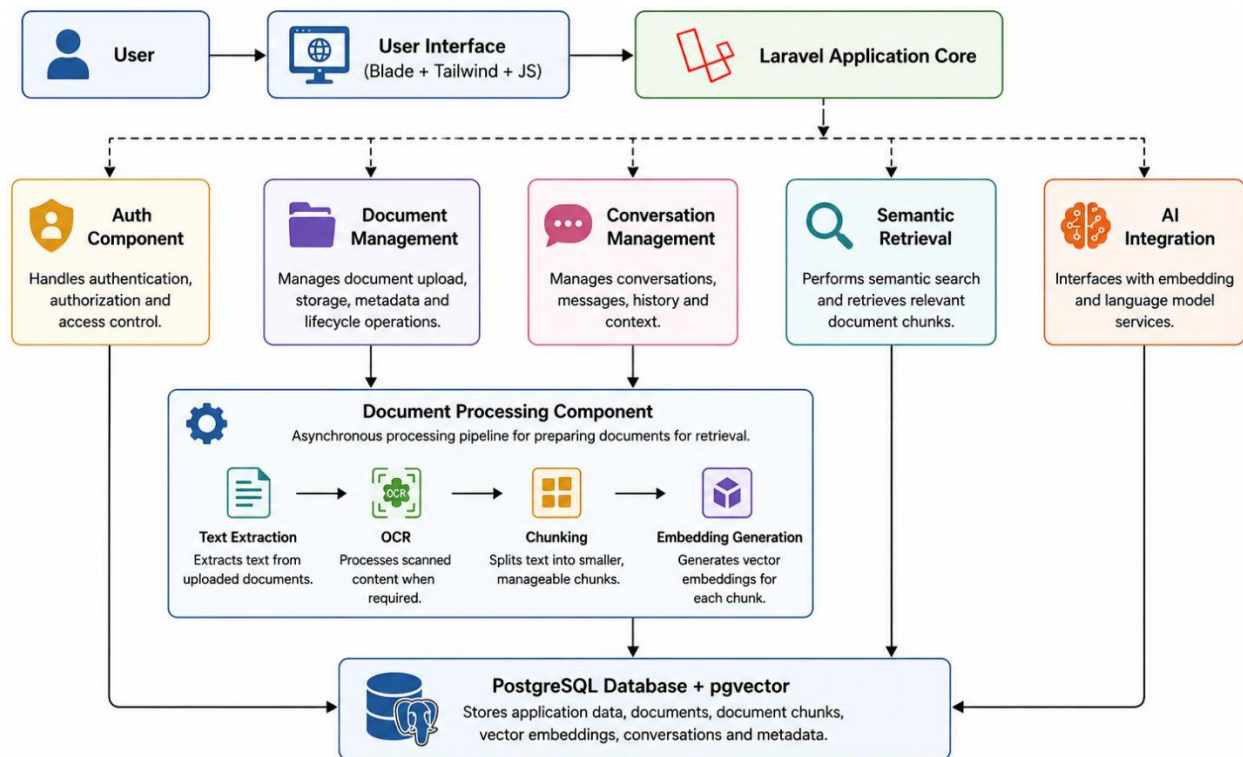


Figure 6.2: UML Component Diagram of the AI-Powered Enterprise RAG System

6.3. Class Diagram

The class diagram presented in Figure 6.3 illustrates the static structure of the AI-powered Retrieval-Augmented Generation (RAG) system by identifying the primary classes, their key attributes, and the relationships between them. While the component diagram describes the interaction between the major software modules, the class diagram focuses on the object-oriented design of the application and the relationships between its core entities and service classes.

The User class represents authenticated users of the application and serves as the starting point of the system's data model. Each user can upload multiple documents and create multiple conversations with the AI assistant. This one-to-many relationship allows every user to maintain an independent collection of documents and conversation history while ensuring proper separation of user data.

Uploaded documents are represented by the Document class, which stores essential information such as the document name, processing status, and file location. During the document processing pipeline, each document is divided into multiple DocumentChunk objects. Every document chunk contains a portion of the extracted text together with its corresponding embedding vector and metadata required for semantic search. Since a single document can generate many chunks, the relationship between Document and DocumentChunk is one-to-many.

The Conversation class represents an individual chat session initiated by a user. Each conversation consists of multiple Message objects that store both user queries and AI-generated responses. This

separation enables the application to preserve the complete conversation history and maintain contextual information throughout an ongoing discussion.

To support conversations involving multiple uploaded documents, the ConversationDocument class functions as an associative entity between Conversation and Document. This many-to-many relationship allows users to select one or more documents as knowledge sources for a conversation while enabling the same document to participate in multiple independent conversations without duplicating stored information.

Besides the core domain entities, the system contains several service classes that implement the application's business logic. The DocumentProcessor service manages document processing operations, including text extraction, preprocessing, and document chunk generation. The EmbeddingService converts both document chunks and user queries into vector embeddings that enable semantic similarity search. The AIChatService coordinates the retrieval workflow by obtaining the most relevant document chunks and communicating with the language model to generate context-aware responses. Separating these responsibilities into dedicated service classes improves modularity while keeping the business logic independent from the persistent data model.

Overall, the class diagram demonstrates how the system combines object-oriented design principles with a modular service architecture. The clear separation between persistent entities and service classes improves maintainability, simplifies future extensions, and provides a well-structured foundation for document processing, semantic retrieval, and AI-assisted knowledge retrieval within the enterprise RAG system.

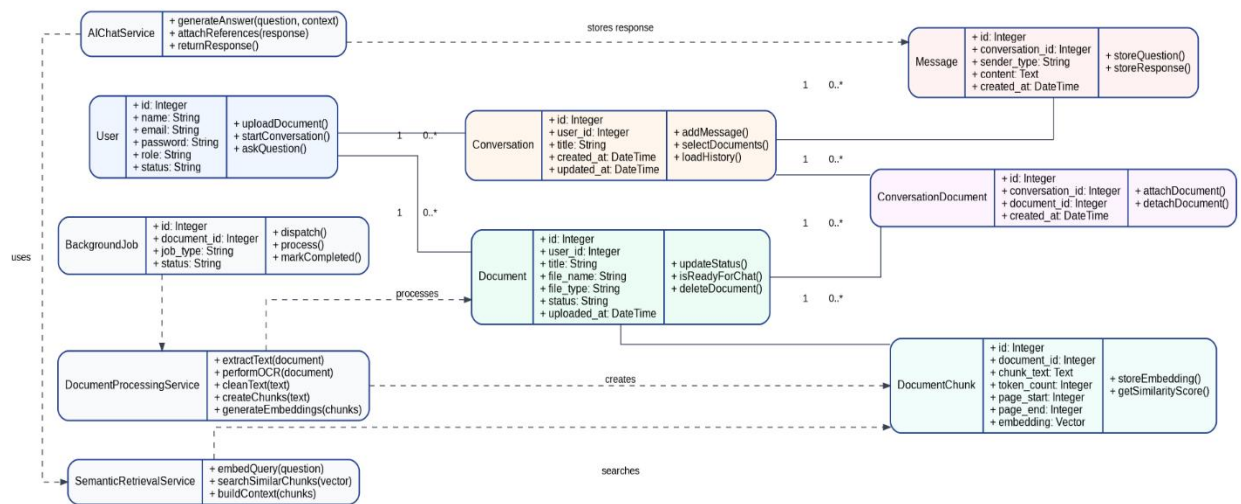


Figure 6.3: UML Class Diagram of the AI-Powered Enterprise RAG System

6.4. Sequence Diagrams

Sequence diagrams describe the dynamic interactions between the different components of the AI-powered Retrieval-Augmented Generation (RAG) system during the execution of key business processes. Unlike the class diagram, which illustrates the static structure of the application, sequence diagrams

emphasize the chronological order in which system components communicate to complete a specific task.

The developed system contains two primary workflows that represent its core functionality. The first workflow describes how uploaded documents are processed and prepared for semantic retrieval, while the second workflow illustrates how user questions are answered through the Retrieval-Augmented Generation process. Together, these sequence diagrams provide a comprehensive view of the interaction between the user interface, application services, document processing pipeline, database, and artificial intelligence services during normal system operation.

6.4.1. Document Upload & Processing

The document upload and processing workflow begins when a user uploads a supported document through the web interface. After receiving the upload request, the Laravel application validates the document and stores the original file together with its associated metadata. Once the upload has been completed successfully, the application initiates the background document processing workflow.

The document processing component performs a sequence of operations to prepare the document for semantic retrieval. First, textual content is extracted from the uploaded document. If the document contains scanned pages rather than searchable text, the OCR component is used to convert the images into machine-readable text. The extracted content is subsequently cleaned and divided into multiple semantic chunks that preserve contextual meaning while remaining suitable for efficient retrieval.

Each generated document chunk is then forwarded to the embedding service, which converts the textual content into numerical vector representations. These embeddings, together with the document chunks and related metadata, are stored within the PostgreSQL database enhanced with vector storage capabilities. Throughout this process, the application continuously updates the processing status, allowing users to monitor the progress through the user interface.

Once all processing stages have been completed successfully, the document status is updated to indicate that it is ready for conversational querying. From this point onward, the processed document becomes available as a knowledge source for semantic retrieval during user conversations.

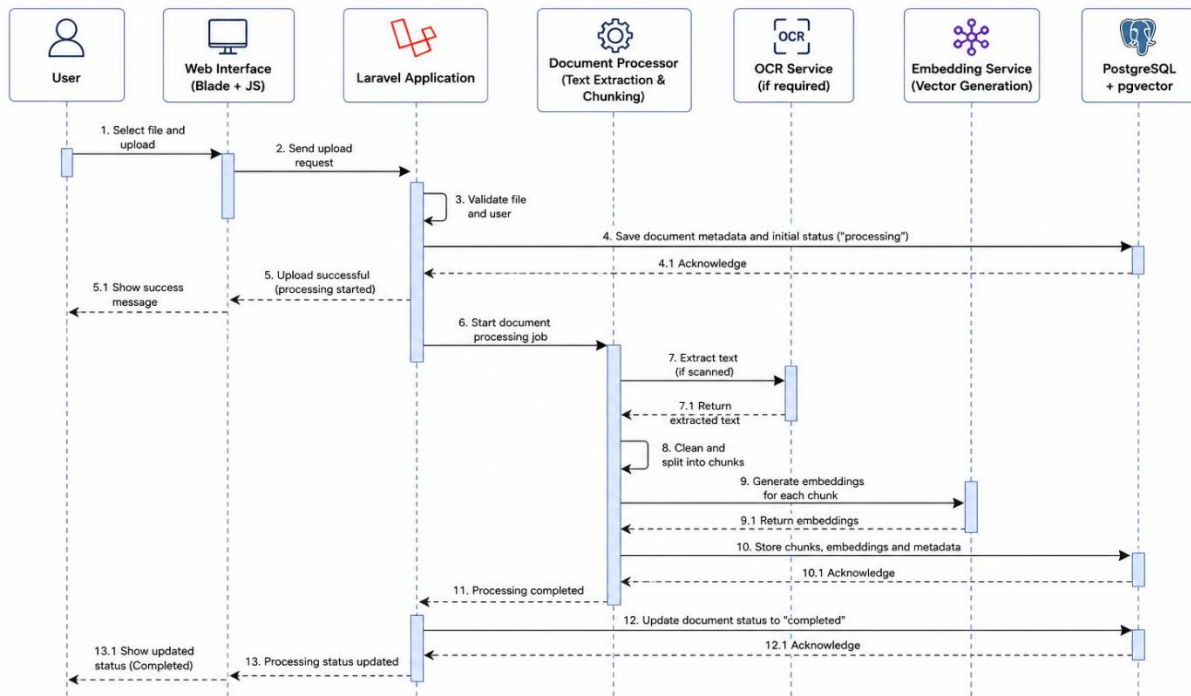


Figure 6.4: Sequence Diagram for Document Upload and Processing Workflow

6.4.2. Question Answering Workflow

The question-answering workflow begins when a user submits a natural language question within an existing conversation. The user interface forwards the request to the Laravel application together with the identifiers of the selected knowledge sources associated with the conversation.

The application first generates a vector embedding for the user's question using the embedding service. This query embedding is then compared with the stored document embeddings through semantic similarity search performed on the PostgreSQL vector database. Based on the similarity scores, the system retrieves the document chunks that are most relevant to the user's query.

The retrieved document context, together with the user's original question, is subsequently forwarded to the language model. Rather than processing entire documents, the language model receives only the selected document fragments that are considered most relevant by the retrieval mechanism. This Retrieval-Augmented Generation approach improves response relevance while reducing unnecessary information processing.

After the language model generates a context-aware response, the Laravel application stores both the user's question and the generated answer within the conversation history. Any associated document references are also preserved to improve transparency and allow users to verify the origin of the retrieved information. Finally, the completed response is returned to the user interface, where it is displayed within the conversation together with the corresponding document references.

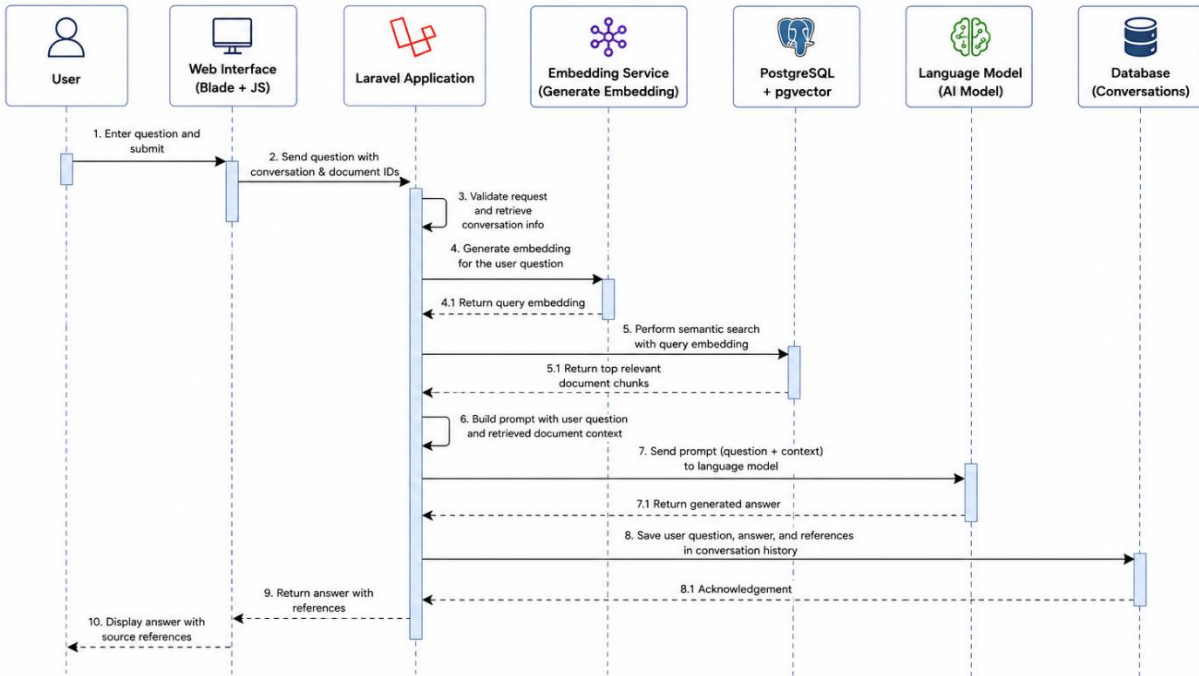


Figure 6.5: Sequence Diagram for AI-Assisted Question Answering Workflow